

SQL Basics

Assignment Questions



1. Create a table called employees with the following structure

- ✓ emp_id (integer, should not be NULL and should be a primary key)
- ✓ emp_name (text, should not be NULL)
- ✓ age (integer, should have a check constraint to ensure the age is at least 18)
- ✓ email (text, should be unique for each employee)
- ✓ salary (decimal, with a default value of 30,000).

Write the SQL query to create the above table with all constraints.

```
CREATE TABLE employees (  
    emp_id INT NOT NULL PRIMARY KEY,  
    emp_name VARCHAR(100) NOT NULL,  
    age INT CHECK (age >= 18),  
    email VARCHAR(100) UNIQUE,  
    salary DECIMAL(10, 2) DEFAULT 30000  
);
```

DESC employees;

Result Grid						
		Filter Rows:			Export:	Wra
	Field	Type	Null	Key	Default	Extra
▶	emp_id	int	NO	PRI	NULL	
	emp_name	varchar(100)	NO		NULL	
	age	int	YES		NULL	
	email	varchar(100)	YES	UNI	NULL	
	salary	decimal(10,2)	YES		30000.00	

2. Explain the purpose of constraints and how they help maintain data integrity in a database. Provide examples of common types of constraints.

Purpose of Constraints

Constraints are rules enforced on data in database tables to ensure **accuracy**, **validity**, and **consistency**. They help maintain **data integrity** by:

- ☐ Preventing invalid data entry
- ☐ Enforcing business rules
- ☐ Avoiding data duplication or conflicts

Common Types of Constraints with Examples

Constraint	Purpose	Example
PRIMARY KEY	Ensures each row is uniquely identifiable	emp_id INT PRIMARY KEY
NOT NULL	Prevents a column from having NULL (empty) values	emp_name VARCHAR(50) NOT NULL
UNIQUE	Ensures all values in a column are different	email VARCHAR(100) UNIQUE
CHECK	Ensures values meet a specific condition	age INT CHECK (age >= 18)
DEFAULT	Sets a default value if none is provided	salary DECIMAL DEFAULT 30000
FOREIGN KEY	Ensures value exists in another table (referential integrity)	customer_id INT, FOREIGN KEY (customer_id) REFERENCES customer(customer_id)

3. Why would you apply the NOT NULL constraint to a column? Can a primary key contain NULL values? Justify your answer.

📌 Purpose of NOT NULL Constraint

The NOT NULL constraint ensures that a column must have a value — it prevents the insertion of NULL (empty or missing) data.

◆ Benefits:

- Enforces mandatory fields (e.g., names, dates)
- Improves data integrity and consistency
- Prevents incomplete records

❓ Can a Primary Key Contain NULL?

No, a primary key can never contain NULL values.

🔍 Why?

A primary key uniquely identifies each row in a table. NULL means “unknown” — you can’t uniquely identify a record with an unknown value. By definition, primary keys must be both NOT NULL and UNIQUE

Example

```
CREATE TABLE employees (  
    emp_id INT NOT NULL PRIMARY KEY,  
    emp_name VARCHAR(100) NOT NULL,  
    age INT CHECK (age >= 18),  
    email VARCHAR(100) UNIQUE,  
    salary DECIMAL(10, 2) DEFAULT 30000  
);
```

4. Explain the steps and SQL commands used to add or remove constraints on an existing table. Provide an example for both adding and removing a constraint.

In SQL, you can use ALTER TABLE to add or remove constraints from an existing table.

◆ A. Add a Constraint

- Example: Add a UNIQUE constraint to email column

```
ALTER TABLE employees  
ADD CONSTRAINT unique_email UNIQUE (email);
```

◆ B. Remove a Constraint

- Example: Remove the UNIQUE constraint on email .

```
ALTER TABLE employees  
DROP INDEX unique_email;
```

5. Explain the consequences of attempting to insert, update, or delete data in a way that violates constraints. Provide an example of an error message that might occur when violating a constraint.

🔗 What Happens When Constraints Are Violated?

When you attempt to insert, update, or delete data that violates a defined constraint, the database system will reject the operation and return an error.

This protects the integrity and consistency of your data.

◆ Examples of Violating Constraints:

1. NOT NULL Constraint

```
INSERT INTO employees (emp_id, emp_name) VALUES (102, NULL);
```

● Error:

ERROR 1048 (23000):
Column 'emp_name' cannot be null

2. UNIQUE Constraint

```
INSERT INTO employees (emp_id, emp_name, age, email)  
VALUES (103, 'Jane Smith', 30, 'alice.johnson@example.com');
```

● Error:

ERROR 1062 (23000): Duplicate entry
'alice.johnson@example.com' for key
'unique_email'

3. CHECK Constraint

```
INSERT INTO employees (emp_id, emp_name, age, email)  
VALUES (104, 'Tom Lee', 15, 'tom@example.com');
```

● Error:

ERROR 3819 (HY000): Check
constraint 'employees_chk_1' is
violated.

6. You created a products table without constraints as follows:

```
CREATE TABLE products (  
  product_id INT,  
  product_name VARCHAR(50),  
  price DECIMAL(10, 2));
```

Now, you realise that The product_id should be a primary key The price should have a default value of 50.00

Modify the products table to add constraints

Add primary key to product_id

Set default value of 50.00 for price

```
CREATE TABLE products (  
  product_id INT,  
  product_name VARCHAR(50),  
  price DECIMAL(10, 2)  
);  
  
ALTER TABLE products  
ADD PRIMARY KEY (product_id);  
  
ALTER TABLE products  
MODIFY price DECIMAL(10, 2) DEFAULT 50.00;
```

7. You have two tables:

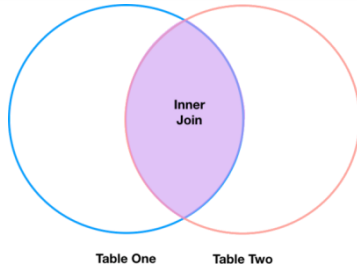
Write a query to fetch the student_name and class_name for each student using an INNER JOIN.

Students:

student_id	student_name	class_id
1	Alice	101
2	Bob	102
3	Charlie	101

Classes:

class_id	class_name
101	Math
102	Science
103	History



student_name and class_name using INNER JOIN



SELECT

s.student_name,
c.class_name

FROM

Students s

INNER JOIN

Classes c **ON** s.class_id = c.class_id;

8. Consider the following three tables:

Write a query that shows all order_id, customer_name, and product_name, ensuring that all products are listed even if they are not associated with an order Hint: (use INNER JOIN and LEFT JOIN)

Orders:

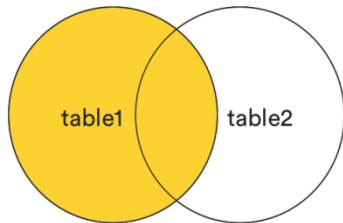
order_id	order_date	customer_id
1	2024-01-01	101
2	2024-01-03	102

Customers:

customer_id	customer_name
101	Alice
102	Bob

Products:

product_id	product_name	order_id
1	Laptop	1
2	Phone	NULL



LEFT JOIN

SELECT

p.order_id,
c.customer_name,
p.product_name

FROM

Products p

LEFT JOIN Orders o ON p.order_id = o.order_id

LEFT JOIN Customers c ON o.customer_id = c.customer_id;

Why This Works:

LEFT JOIN ensures all products are listed — including those with no order_id

INNER JOIN replaced with LEFT JOIN on Customers because an order might be missing

9. Given the following tables:

Write a query to find the total sales amount for each product using an INNER JOIN and the SUM() function

◦ Sales:

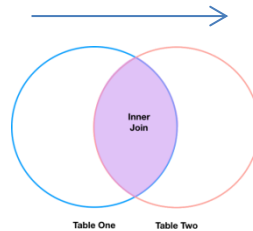
sale_id	product_id	amount
1	101	500
2	102	300
3	101	700

◦ Products:

product_id	product_name
101	Laptop
102	Phone

SQL INPUT

```
SELECT
    p.product_name,
    SUM(s.amount) AS total_sales
FROM
    Products p
INNER JOIN
    Sales s ON p.product_id = s.product_id
GROUP BY
    p.product_name;
```



SQL OUTPUT

product_name	total_sales
--------------	-------------

Laptop	1200
--------	------

Phone	300
-------	-----

10. You are given three tables:

Write a query to display the `order_id`, `customer_name`, and the quantity of products ordered by each customer using an `INNER JOIN` between all three tables.

Orders:

order_id	order_date	customer_id
1	2024-01-02	1
2	2024-01-05	2

Customers:

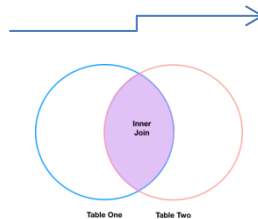
customer_id	customer_name
1	Alice
2	Bob

Order_Details:

order_id	product_id	quantity
1	101	2
1	102	1
2	101	3

SQL INPUT

```
SELECT
  o.order_id,
  c.customer_name,
  od.quantity
FROM
  Orders o
INNER JOIN Customers c ON o.customer_id = c.customer_id
INNER JOIN Order_Details od ON o.order_id = od.order_id;
```



SQL OUTPUT

order_id	customer_name	quantity
1	Alice	2
1	Alice	1
2	Bob	3

THANKS FOR THE ASSIGNMENT